

ELE432 Advanced Digital Design Homework 2

- 1) I spent 5 hours on this homework.
- 2) "aludec" and "instrdec" were given in the homework 2 pdf.

RTL Schematic is given below.

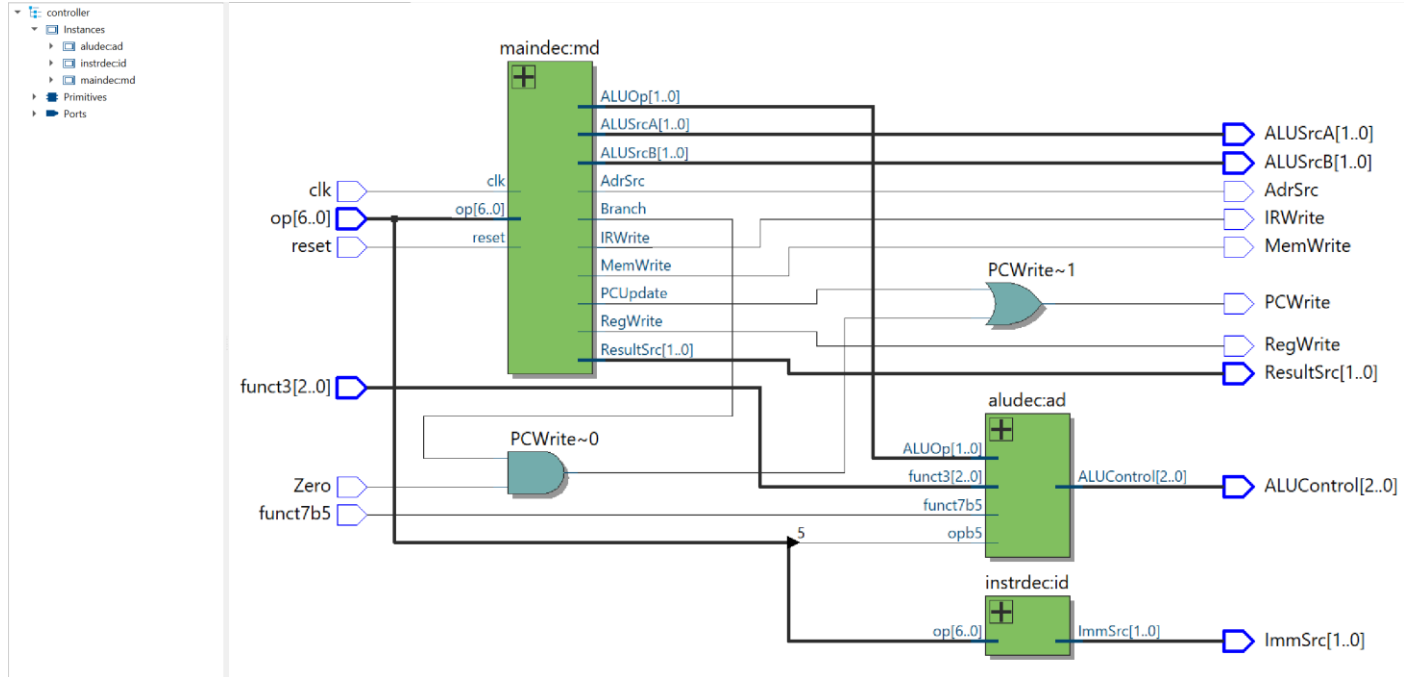


Figure 1 RTL Schematic

File: controller.sv

```

/*
  Egemen Celik 2220357034
  ELE432 Homework 2 : Multicycle_Processor_Controller
  controller.sv
  code modified from Harris&Harris book
*/

module controller (
  input logic clk,
  input logic reset,
  input logic [6:0] op,
  input logic [2:0] funct3,
  input logic funct7b5,
  input logic Zero,
  output logic [1:0] ImmSrc,
  output logic [1:0] ALUSrcA, ALUSrcB,
  output logic [1:0] ResultSrc,
  output logic AdrSrc,
  output logic [2:0] ALUControl,
  output logic IRWrite, PCWrite,
  output logic RegWrite, MemWrite
);
  // internal wires
  logic Branch;
  logic PCUpdate;
  logic [1:0] ALUOp;

  assign PCWrite = (Zero & Branch) | PCUpdate;

```

```

// decoder logic
maindec md (
  clk,
  reset,
  op,
  Branch,
  PCUpdate,
  ALUSrcA,
  ALUSrcB,
  ResultSrc,
  AdrSrc,
  IRWrite,
  RegWrite,
  MemWrite,
  ALUOp
);

aludec ad (
  op[5],
  funct3,
  funct7b5,
  ALUOp,
  ALUControl
);

instrdec id (
  op,
  ImmSrc
);

endmodule

```

File: maindec.sv

```

/*      Egemen Celik 2220357034
      ELE432 HomeWork 2 : Multicycle_Processor_Controller
      maindec.sv      */
module maindec (
    input logic clk,
    input logic reset,
    input logic [6:0] op,
    output logic Branch,
    output logic PCUpdate,
    output logic [1:0] ALUSrcA,
    output logic [1:0] ALUSrcB,
    output logic [1:0] ResultSrc,
    output logic AddrSrc,
    output logic IRWrite,
    output logic RegWrite,
    output logic MemWrite,
    output logic [1:0] ALUOp
);
// State encoding
typedef enum logic [3:0] {
    S0_FETCH = 4'b0000,
    S1_DECODE = 4'b0001,
    S2_MEMADR = 4'b0010,
    S3_MEMREAD = 4'b0011,
    S4_MEMWB = 4'b0100,
    S5_MEMWRITE = 4'b0101,
    S6_EXECUTER = 4'b0110,
    S7_ALUWB = 4'b0111,
    S8_EXECUTEI = 4'b1000,
    S9_JAL = 4'b1001,
    S10_BEQ = 4'b1010
} state_t;

// State variables
state_t current_state, next_state;

// Opcode definitions
localparam logic [6:0] OP_LW = 7'b0000011;
localparam logic [6:0] OP_SW = 7'b0100011;
localparam logic [6:0] OP_RTYPE = 7'b0110011;
localparam logic [6:0] OP_ITYPE = 7'b0010011;
localparam logic [6:0] OP_JAL = 7'b1101111;
localparam logic [6:0] OP_BEQ = 7'b1100011;

// Sequential State transition logic
always_ff @(posedge clk or posedge reset) begin : Sequential_State
    if (reset) current_state <= S0_FETCH;
    else current_state <= next_state;
end

// Combinational State transition logic
always_comb begin : Combinational_State
    // Default values
    next_state = current_state;

    case (current_state)
        S0_FETCH: next_state = S1_DECODE;
        S1_DECODE:
            case (op)
                OP_LW, OP_SW: next_state = S2_MEMADR;
                OP_RTYPE: next_state = S6_EXECUTER;
                OP_ITYPE: next_state = S8_EXECUTEI;
                OP_JAL: next_state = S9_JAL;
                OP_BEQ: next_state = S10_BEQ;
                default: next_state = S0_FETCH;
            endcase
        S2_MEMADR:
            case (op)
                OP_LW: next_state = S3_MEMREAD;
                OP_SW: next_state = S5_MEMWRITE;
                default: next_state = S0_FETCH;
            endcase
        S3_MEMREAD: next_state = S4_MEMWB;
        S4_MEMWB, S5_MEMWRITE, S7_ALUWB, S10_BEQ: next_state = S0_FETCH;
        S6_EXECUTER, S8_EXECUTEI, S9_JAL: next_state = S7_ALUWB;
        default: next_state = S0_FETCH;
    endcase
end
end

```

```

// Output logic
always_comb begin : Outputs
    // Default values
    Branch = 1'b0;
    PCUpdate = 1'b0;
    ALUSrcA = 2'b00;
    ALUSrcB = 2'b00;
    ResultSrc = 2'b00;
    AddrSrc = 1'b0;
    IRWrite = 1'b0;
    RegWrite = 1'b0;
    MemWrite = 1'b0;
    ALUOp = 2'b00;

    case (current_state)
        S0_FETCH: begin
            AddrSrc = 1'b0;
            IRWrite = 1'b1;
            ALUSrcA = 2'b00;
            ALUSrcB = 2'b10;
            ALUOp = 2'b00;
            ResultSrc = 2'b10;
            PCUpdate = 1'b1;
        end
        S1_DECODE: begin
            ALUSrcA = 2'b01;
            ALUSrcB = 2'b01;
            ALUOp = 2'b00;
        end
        S2_MEMADR: begin
            ALUSrcA = 2'b10;
            ALUSrcB = 2'b01;
            ALUOp = 2'b00;
        end
        S3_MEMREAD: begin
            ResultSrc = 2'b00;
            AddrSrc = 1'b1;
        end
        S4_MEMWB: begin
            ResultSrc = 2'b01;
            RegWrite = 1'b1;
        end
        S5_MEMWRITE: begin
            ResultSrc = 2'b00;
            AddrSrc = 1'b1;
            MemWrite = 1'b1;
        end
        S6_EXECUTER: begin
            ALUSrcA = 2'b10;
            ALUSrcB = 2'b00;
            ALUOp = 2'b10;
        end
        S7_ALUWB: begin
            ResultSrc = 2'b00;
            RegWrite = 1'b1;
        end
        S8_EXECUTEI: begin
            ALUSrcA = 2'b10;
            ALUSrcB = 2'b01;
            ALUOp = 2'b10;
        end
        S9_JAL: begin
            ALUSrcA = 2'b01;
            ALUSrcB = 2'b10;
            ALUOp = 2'b00;
            ResultSrc = 2'b00;
            PCUpdate = 1'b1;
        end
        S10_BEQ: begin
            ALUSrcA = 2'b10;
            ALUSrcB = 2'b00;
            ALUOp = 2'b01;
            ResultSrc = 2'b00;
            Branch = 1'b1;
        end
    endcase
end
endmodule

```

3) The controller.tv file was using a RISC-V standard different that the one in the Harris & Harris book and the homework file. I updated the expected ALUControl bits to match the Harris & Harris's description. The updated file can be found in the GitHub repository.

Example:

old		updated	
//op[6:0]_funct3[2:0]_funct7b5_Zero_ImmSrc[1:0]_ALUSrcA[1:0]_ALUSrcB[1:0]_ResultSrc[1:0]_AdrSrc_ALUControl[2:0]_IRWrite_PCWrite_RegWrite_MemWrite			
// add			
0110011_000_0_0_XX_00_10_10_0_010_1_1_0_0	// Fetch	0110011_000_0_0_XX_00_10_10_0_000_1_1_0_0	// Fetch
0110011_000_0_0_XX_01_01_00_0_010_0_0_0_0	// Decode	0110011_000_0_0_XX_01_01_00_0_000_0_0_0_0	// Decode
0110011_000_0_0_XX_10_00_00_0_010_0_0_0_0	// ExecuteR	0110011_000_0_0_XX_10_00_00_0_000_0_0_0_0	// ExecuteR
0110011_000_0_0_XX_00_00_00_0_010_0_0_1_0	// ALUWB	0110011_000_0_0_XX_00_00_00_0_000_0_0_1_0	// ALUWB

Testbench Waveform Output:



Questa Transcript Output:

```

Transcript
VSIM 18> run -all
# 40 tests completed with 0 errors
# ** Note: $stop : C:/Projects/quartus/Multicycle_Processor_Controller/controller_testbench.sv(81)
# Time: 415 ps Iteration: 1 Instance: /testbench
# Break in Module testbench at C:/Projects/quartus/Multicycle_Processor_Controller/controller_testbench.sv line 81

```

As you can see after the issue with the ALUControl bits is resolved there are no errors and everything works as expected.

4) The whole Quartus project directory along with all of the files of interest are uploaded inside separate folders along with this file in both Word and PDF forms.

GitHub link: https://github.com/20egemen04/Multicycle_Processor_Controller